PSTAT220A Homework 3

Houpu Li

2026-11-15

Problem 1

Consider the data on teen gambling in `teengamb` from the `faraway` package. Fit a simple linear regression model with the expenditure on gambling as the response and income as the covariate:

(a) Check the constant variance assumption for errors. What do you conclude?
(b) Assume that constant variance assumption is violated and that instead the residual variance grows linearly with income. Fit a weighted least squares to account for this fact. Report the coefficient and confidence interval.
(c) Construct a bootstrap confidence interval of the regression coefficients and of the residual variance (assuming an income of 1).
(d) Make a plot showing the prediction intervals (again assuming residual variance grows linearly with income). The plot should have `income` on the x-axis, `gamble` on the y-axis, and prediction bands for each point.
(e) Fit a robust regression model using `rlm`. Compare the coefficients from your weighted least squares fit.
(f) Use `optim` to a fit a t-regression. That is, fit the linear model assuming iid residuals with a t_3 distribution. Construct a bootstrap confidence interval of the regression coefficients.

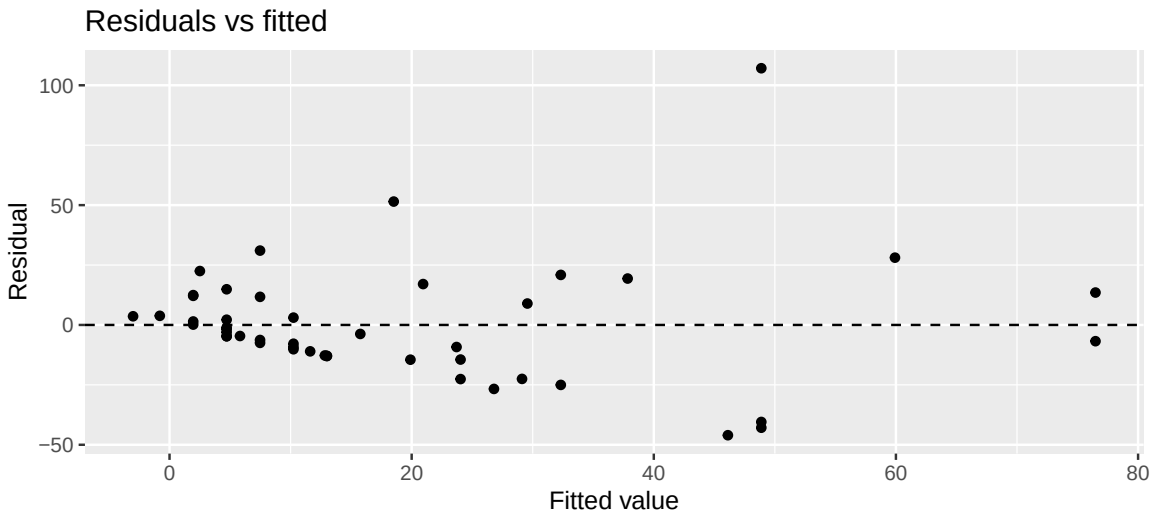
Solution

(a)

```
  sex status income verbal gamble
1   1     51   2.00      8    0.0
2   1     28   2.50      8    0.0
3   1     37   2.00      6    0.0
4   1     28   7.00      4    7.3
5   1     65   2.00      8   19.6
6   1     61   3.47      6    0.1
```

```r
fit_ols <- lm(gamble ~ income, data = teengamb)

ggplot(tibble(fitted = fitted(fit_ols), resid = residuals(fit_ols)),
       aes(fitted, resid)) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "Fitted value", y = "Residual", title = "Residuals vs fitted")
```




```r
bp <- lmtest::bptest(fit_ols)
```

From the residuals vs fitted plot, the constant variance assumption does not hold: as the fitted value increases, the residuals spread out more widely around zero, forming a funnel shape. The error variance increases with income.

(b) Accoding to the background, "Assume that the residual variance grows linearly with income". Thus, I can find $\text{Var}(\epsilon_i) = \sigma^2 \cdot \text{income}_i$, and the weights are $w_i = 1/\text{income}_i$.

```r
fit_wls <- lm(gamble ~ income, data = teengamb, weights = 1 / income)
make_tab <- function(fit, model) {
                                    tibble(Model = model,
                                           Term = names(coef(fit)),
                                           Estimate = coef(fit),
                                           SE = summary(fit)$coefficients[, 2],
                                           `2.5 %` = confint(fit)[, 1],
                                           `97.5 %` = confint(fit)[, 2])
                                  }

bind_rows(make_tab(fit_ols, "OLS"), make_tab(fit_wls, "WLS")) |>
  knitr::kable(digits = 3, caption = "OLS and WLS estimates with 95% confidence intervals")
```

Table 1: OLS and WLS estimates with 95% confidence intervals

Model	Term	Estimate	SE	2.5 %	97.5 %
OLS	(Intercept)	-6.325	6.030	-18.469	5.820
OLS	income	5.520	1.036	3.434	7.607
WLS	(Intercept)	-2.333	3.780	-9.947	5.281
WLS	income	4.661	1.054	2.538	6.783

I assume the observations with higher income are noisier and receive less weight.

The WLS slope is 4.66, with 95% CI (2.54, 6.78), smaller than the OLS slope of 5.52. The WLS intercept is closer to 0 and has a smaller standard error (3.78 vs 6.03). The OLS standard errors assume constant variance and are not valid here, so the OLS intervals should not be trusted.

(c)

I use bootstrap to resample rows and refit the WLS model, finally record the coefficients and $\hat{\sigma}^2$ (the residual variance at income = 1).

```
set.seed(10)
B <- 2000
boot_wls <- replicate(B, {
  d <- teengamb[sample(nrow(teengamb), replace = TRUE), ]
  f <- lm(gamble ~ income, data = d, weights = 1 / income)
  c(coef(f), sigma2 = sigma(f)^2)
})
boot_ci <- t(apply(boot_wls, 1, quantile, probs = c(0.025, 0.975)))
boot_ci <- cbind(Estimate = c(coef(fit_wls), sigma(fit_wls)^2), boot_ci)
rownames(boot_ci) <- c("(Intercept)", "income", "sigma^2 (income = 1)")
knitr::kable(boot_ci, digits = 3, caption = "Bootstrap 95% percentile intervals (B = 2000)")
```

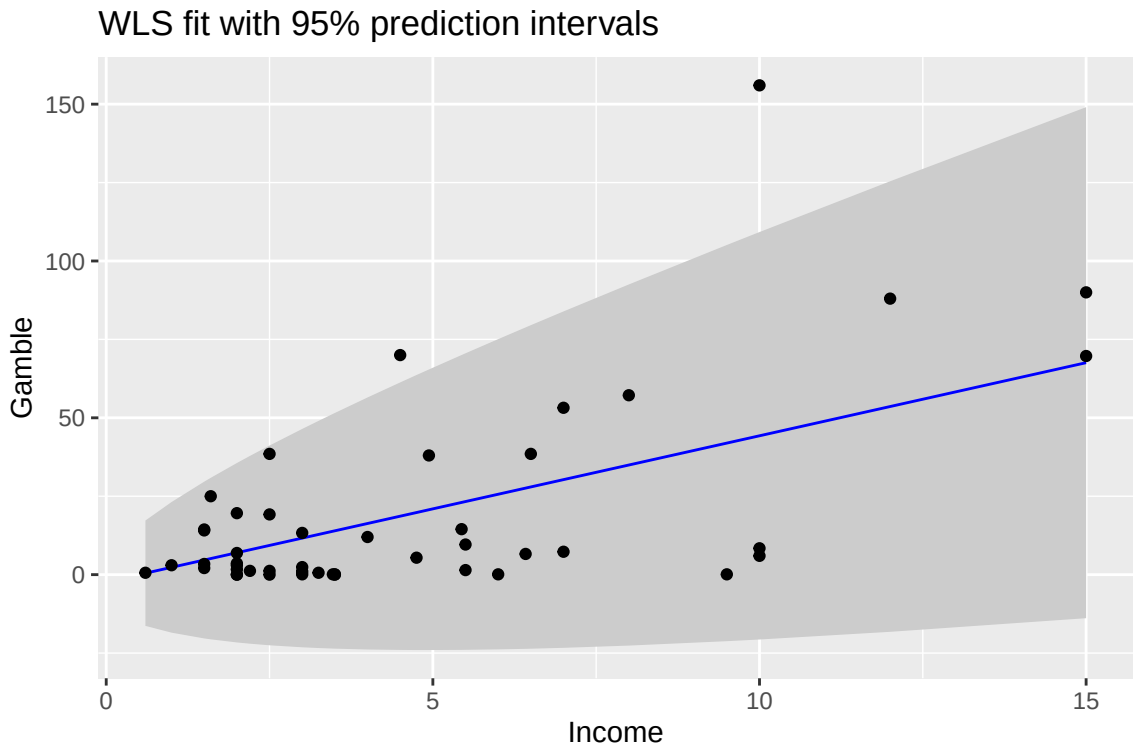
Table 2: Bootstrap 95% percentile intervals (B = 2000)

	Estimate	2.5%	97.5%
(Intercept)	-2.333	-9.021	4.182
income	4.661	2.178	7.329
sigma^2 (income = 1)	97.602	44.639	150.247

(d)

```r
pred <- predict(fit_wls, newdata = teengamb, interval = "prediction", weights = 1 / teengamb
plot_df <- cbind(teengamb, pred)

ggplot(plot_df, aes(x = income)) +
  geom_ribbon(aes(ymin = lwr, ymax = upr), fill = "grey80") +
  geom_line(aes(y = fit), color = "blue") +
  geom_point(aes(y = gamble)) +
  labs(x = "Income", y = "Gamble", title = "WLS fit with 95% prediction intervals")
```



(e)

```r
fit_rlm <- MASS::rlm(gamble ~ income, data = teengamb)
coef_cmp <- rbind(OLS = coef(fit_ols), WLS = coef(fit_wls), RLM = coef(fit_rlm))
se_cmp <- rbind(OLS = summary(fit_ols)$coef[, 2], WLS = summary(fit_wls)$coef[, 2],
                RLM = summary(fit_rlm)$coef[, 2])
colnames(se_cmp) <- paste("SE", colnames(se_cmp))
knitr::kable(cbind(coef_cmp, se_cmp), digits = 3, caption = "OLS, WLS and robust (Huber) est
```

Table 3: OLS, WLS and robust (Huber) estimates

	(Intercept)	income	SE (Intercept)	SE income
OLS	-6.325	5.520	6.03	1.036
WLS	-2.333	4.661	3.78	1.054
RLM	-5.942	5.017	4.51	0.775

The rlm slope (5.02) lies between the OLS slope (5.52) and the WLS slope (4.66). WLS down-weights teens with high income, while rlm down-weights teens with large residuals. All three methods agree that gambling increases with income.Both methods give a smaller slope than OLS, because a few high-income teens with very large expenditures pull the OLS slope up.

(f)

```
# Negative log-likelihood of a linear model with t3 errors
nll_t <- function(par, x, y) {
  s <- exp(par[3])                      # scale, kept positive
  r <- (y - par[1] - par[2] * x) / s    # standardized residuals
  -sum(dt(r, df = 3, log = TRUE) - log(s))
}

# Fit the t3 regression with optim, starting from the OLS fit
fit_t <- function(d) {
  start <- c(coef(lm(gamble ~ income, data = d)), log(sd(d$gamble)))
  optim(start, nll_t, x = d$income, y = d$gamble)$par[1:2]
}

fit_t(teengamb)
```

```
(Intercept)      income
  -6.043566    4.651753
```

```
# Bootstrap
set.seed(10)
boot_t <- replicate(2000, fit_t(teengamb[sample(nrow(teengamb), replace = TRUE), ]))
t(apply(boot_t, 1, quantile, probs = c(0.025, 0.975)))
```

```
                  2.5%    97.5%
(Intercept) -12.4797176 6.741900
income        0.1512807 6.994874
```

The t3 regression gives a slope of 4.65, close to the WLS slope and smaller than the OLS slope (5.52). The bootstrap 95% interval for the slope is (0.15, 6.99). It excludes 0, so income has a positive effect.

Problem 2

Standard multiple testing guarantees are given assuming independent tests. In general, test statistics for regression coefficients are correlated.

(a) Assume you have $p = 300$ predictors and $n = 1000$ observations. Let $y = X\beta + \epsilon$ (no intercept), where the first 10 coefficients in β equal 0.2 and the last 90 are 0. Generate $X \sim N(0, 1)$ for each feature. Run a simulation where you generate data from this model many times. Fit the linear model and use both Benjamini-Hochberg and the q-value correction to select discoveries controlling the FDR at 0.05. For each simulated dataset compute 1) the false discovery proportion and 2) the sensitivity (the fraction of the 10 significant predictors that were correctly declared successes). Plot the distribution FDP and sensitivity for both BH and Q-value approaches using histograms or a beeswarm plot or something similar.

(b) Now consider generating data assuming correlated covariates drawn from a multivariate normal, $X \sim N(\mu, \Sigma)$ with Σ an equicorrelation matrix:

$$\Sigma = \begin{pmatrix} 1 & \rho & \cdots & \rho \\ \rho & 1 & \cdots & \rho \\ \vdots & \vdots & \ddots & \vdots \\ \rho & \rho & \cdots & 1 \end{pmatrix} \tag{1}$$

To investigate how problematic positive equicorrelation is for these procedures, repeat part (a) for each of $\rho \in \{0.25, 0.5, 0.75\}$. Plot the distributions as a function of ρ. What can you conclude about how positive equicorrelation amongst the predictors impacts the false discovery proportions? How does it impact sensitivity?

Solution

Simulation design. I set $p = 300$ and $n = 1000$. The first 10 coefficients are 0.2 (true signals) and the other 290 are 0 (nulls). Because I choose the true coefficients ourselves, I know which predictors are real, so I can check how well BH and q-value perform. Part (a) is the case $\rho = 0$ (independent predictors), and part (b) repeats the same experiment with $\rho = 0.25, 0.5, 0.75$.

Step 1: one simulated experiment. I write a function that runs a single experiment for a given ρ and returns four scores(estimate, SE, T-value, P-value). the experiment design as below,

- *Generate X.* `Sigma` is the correlation matrix from the problem: 1 on the diagonal (each predictor has variance 1) and ρ everywhere else. `mvrnorm` draws 1000 rows of X from $N(0, \Sigma)$. When $\rho = 0$, Σ is the identity matrix, which is the independent case in part (a).
- *Generate y.* I set the true coefficients and compute $y = X\beta + \epsilon$ with $\epsilon \sim N(0, 1)$.

- *Fit the model.* I regress y on all 300 predictors without an intercept (`- 1`) and keep the 300 p-values.
- *Select predictors.* BH and q-value each adjust the 300 p-values to control the FDR, and I keep the predictors with adjusted value below 0.05. `which()` returns their index numbers.
- *Score the selection.* Indices 1-10 are true signals and 11-300 are nulls. FDP is the share of selected predictors with index above 10 (false discoveries); if nothing is selected, FDP is 0. Sensitivity is the share of the 10 true signals that were selected.

```
one_sim <- function(rho) {
  # 1. Generate X with pairwise correlation rho
  Sigma <- matrix(rho, 300, 300)
  diag(Sigma) <- 1
  X <- MASS::mvrnorm(1000, rep(0, 300), Sigma)

  # 2. Generate y: first 10 predictors matter, the other 290 do not
  beta <- c(rep(0.2, 10), rep(0, 290))
  y <- X %*% beta + rnorm(1000)

  # 3. Fit the model and get 300 p-values
  pvals <- summary(lm(y ~ X - 1))$coefficients[, 4]

  # 4. Which predictors are selected?
  bh <- which(p.adjust(pvals, method = "BH") < 0.05)
  qv <- which(qvalue(pvals)$qvalues < 0.05)

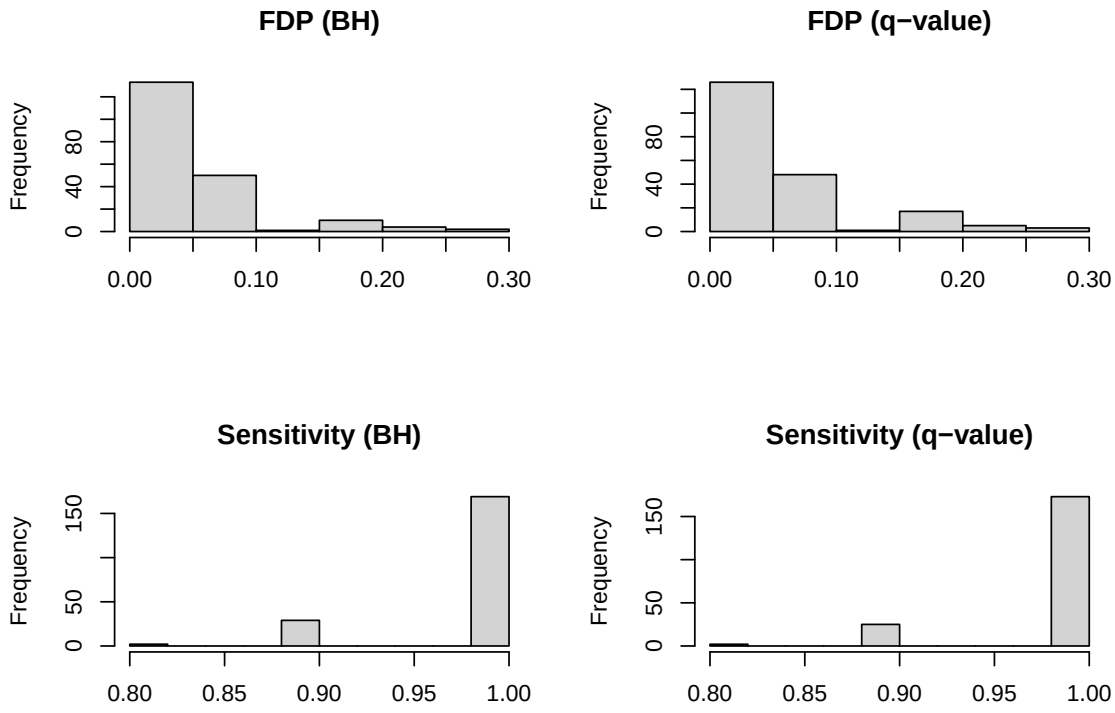
  # 5. Score: FDP = share of selected that are > 10; sensitivity = share of 1-10 found
  fdp <- function(s) if (length(s) == 0) 0 else mean(s > 10)
  c(FDP_BH = fdp(bh), Sens_BH = sum(bh <= 10) / 10,
    FDP_Q  = fdp(qv), Sens_Q  = sum(qv <= 10) / 10)
}
```

Step 2: repeat the experiment. A single experiment depends on the random draw, so I repeat it 200 times for each value of ρ. `replicate(200, one_sim(rho))` runs the experiment 200 times, and `t()` turns the result into 200 rows with four columns. I stack the results for all four values of ρ into one data frame, `results`, with 800 rows = 200*4. `set.seed` makes the results reproducible.

```
set.seed(10)
results <- NULL
for (rho in c(0, 0.25, 0.5, 0.75)) {
  out <- t(replicate(200, one_sim(rho)))
  results <- rbind(results, data.frame(rho = rho, out))
}
```

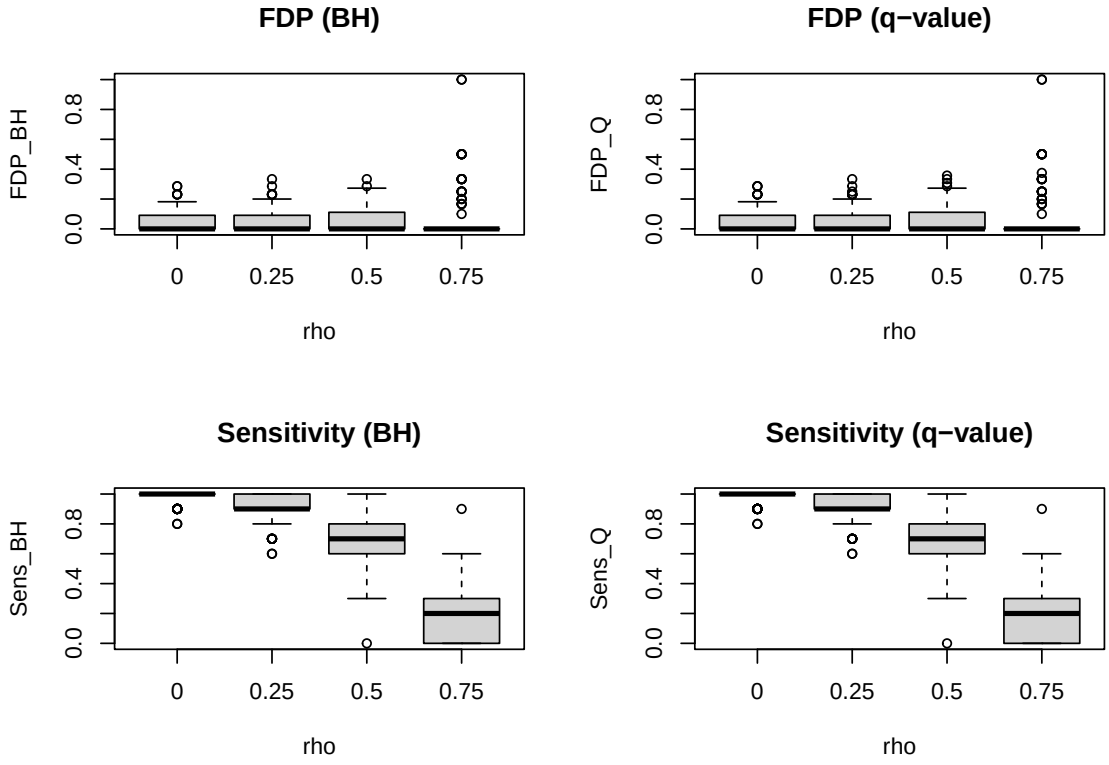
(a) Independent predictors ($\rho = 0$). I keep only the runs with $\rho = 0$ and draw a histogram of each score. Each histogram shows how FDP or sensitivity varies across the 200 simulated datasets.

```
a <- results[results$rho == 0, ]
par(mfrow = c(2, 2))
hist(a$FDP_BH,  main = "FDP (BH)",              xlab = "")
hist(a$FDP_Q,   main = "FDP (q-value)",         xlab = "")
hist(a$Sens_BH, main = "Sensitivity (BH)",      xlab = "")
hist(a$Sens_Q,  main = "Sensitivity (q-value)", xlab = "")
```



(b) Correlated predictors. To see how the scores change with ρ, I draw one boxplot per value of ρ for each score. Each box summarizes the 200 runs at that ρ.

```
par(mfrow = c(2, 2))
boxplot(FDP_BH  ~ rho, data = results, main = "FDP (BH)")
boxplot(FDP_Q   ~ rho, data = results, main = "FDP (q-value)")
boxplot(Sens_BH ~ rho, data = results, main = "Sensitivity (BH)")
boxplot(Sens_Q  ~ rho, data = results, main = "Sensitivity (q-value)")
```



Average scores.

```
aggregate(. ~ rho, data = results, FUN = mean)
```

```
   rho      FDP_BH Sens_BH      FDP_Q Sens_Q
1 0.00 0.03963414  0.9835 0.04709535 0.9855
2 0.25 0.04761447  0.9170 0.05243751 0.9210
3 0.50 0.04549441  0.6755 0.04956047 0.6875
4 0.75 0.04725000  0.1845 0.05245833 0.1910
```

Conclusions.

Part (a) is the baseline. With independent predictors (= 0), both methods work as intended. The mean FDP is 0.040 for BH and 0.047 for q-value, both below the target of 0.05, and the mean sensitivity is about 0.98, so almost all 10 true signals are found.

Impact on the FDP. Predictors correlation does not inflate the FDR. The mean FDP stays between 0.040 and 0.052 for every value of , close to the target of 0.05. However, at = 0.75 the FDP varies more across runs: most runs select very few predictors with FDP close to 0, but more outliers in the boxplot show.

Impact on sensitivity. As increases, both methods still find some of the true signals, but they miss more and more of them. On average, BH finds 9.8 of the 10 true signals at = 0, but only 9.2, 6.8 and 1.8 at = 0.25, 0.5 and 0.75. Combined with the stable FDP, this means the selected predictors are still mostly correct, but the selection becomes incomplete. When predictors are highly correlated, it is hard to tell which one drives y, so the true signals are less often significant and are missed.

Problem 3

Run a regression with `prostate` data in `faraway` library with `lpsa` as the response variable and the other variables as covariates.

(a) Select the best model using each of (1) AIC (either forward or backward selection); (2) best subset selection with cross-validation; and (3) LASSO with cross-validation. How much agreement is there between approaches?

(b) Can you improve the model by adding higher order terms? Can you improve the model by adding multiplicative interaction terms? Use any selection approache of choice to settle on a "final" model.

Solution

(a)

```
      lcavol lweight age      lbph svi      lcp gleason pgg45     lpsa
1 -0.5798185  2.7695  50 -1.386294   0 -1.38629       6     0 -0.43078
2 -0.9942523  3.3196  58 -1.386294   0 -1.38629       6     0 -0.16252
3 -0.5108256  2.6912  74 -1.386294   0 -1.38629       7    20 -0.16252
4 -1.2039728  3.2828  58 -1.386294   0 -1.38629       6     0 -0.16252
5  0.7514161  3.4324  62 -1.386294   0 -1.38629       6     0  0.37156
6 -1.0498221  3.2288  50 -1.386294   0 -1.38629       6     0  0.76547
```

(1) Backward AIC selects: lcavol, lweight, age, lbph, svi.

```
set.seed(10)
full <- lm(lpsa ~ ., data = prostate)
fit_aic <- step(full, direction = "backward", trace = 0)
paste(attr(terms(fit_aic), "term.labels"), collapse = ", ")
```

```
[1] "lcavol, lweight, age, lbph, svi"
```

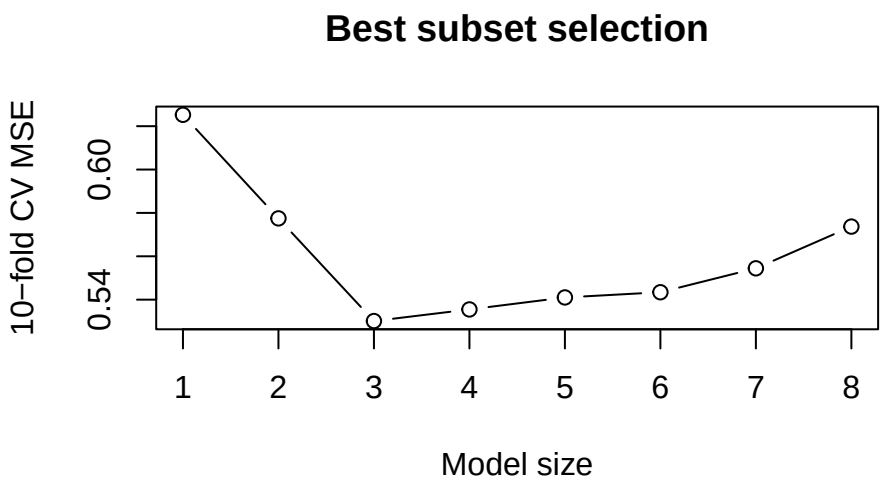
(2) Best subset + 10-fold CV. selects: lcavol, lweight, svi

```r
K <- 10
nobs <- nrow(prostate)
folds <- sample(rep(1:K, length.out = nobs))   # assign each row to one of 10 folds

# Step 1: best combination of variables for each model size (1 to 8)
rs <- regsubsets(lpsa ~ ., data = prostate, nvmax = 8)
vars <- summary(rs)$which[, -1]                # TRUE = variable is in the model

# Step 2: 10-fold CV error for each of the 8 candidate models
cv_mse <- numeric(8)
for (m in 1:8) {
  form <- reformulate(colnames(vars)[vars[m, ]], response = "lpsa")
  errs <- numeric(K)
  for (k in 1:K) {
    fit  <- lm(form, data = prostate[folds != k, ])   # fit on 9 folds
    test <- prostate[folds == k, ]                    # test on the remaining fold
    errs[k] <- mean((test$lpsa - predict(fit, test))^2)
  }
  cv_mse[m] <- mean(errs)
}

# Step 3: pick the size with the smallest CV error
plot(1:8, cv_mse, type = "b", xlab = "Model size", ylab = "10-fold CV MSE",
     main = "Best subset selection")
```



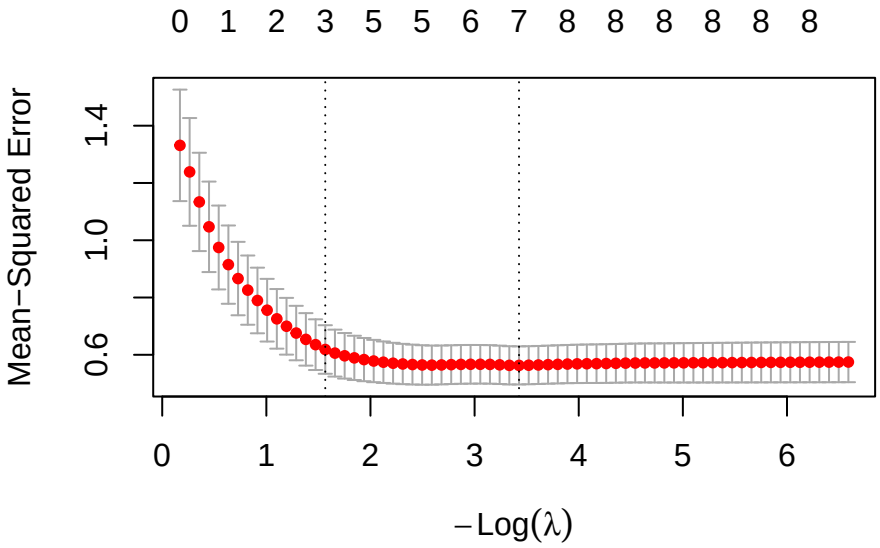

```r
best_size <- which.min(cv_mse)
vars_sub <- colnames(vars)[vars[best_size, ]]
```

```
vars_sub
```

```
[1] "lcavol"  "lweight" "svi"
```

(3) LASSO + 10-fold CV. selects: lcavol, lweight, svi

```
X <- model.matrix(lpsa ~ ., prostate)[, -1]
cv_lasso <- cv.glmnet(X, prostate$lpsa, alpha = 1, foldid = folds)
plot(cv_lasso)
```




```
nz <- function(s) {
  cf <- coef(cv_lasso, s = s)
  rownames(cf)[as.vector(cf != 0)][-1]
}
vars_lasso_min <- nz("lambda.min")
vars_lasso_1se <- nz("lambda.1se")
vars_lasso_1se
```

```
[1] "lcavol"  "lweight" "svi"
```

```r
all_vars <- colnames(X)
sel <- list(`AIC (backward)` = attr(terms(fit_aic), "term.labels"),
            `Best subset (CV)` = vars_sub,
            `LASSO (1-SE)` = vars_lasso_1se)
agree <- sapply(sel, \(v) ifelse(all_vars %in% v, "x", ""))
rownames(agree) <- all_vars
knitr::kable(agree, align = "c", caption = "Variables selected by each method")
```

Table 4: Variables selected by each method

	AIC (backward)	Best subset (CV)	LASSO (1-SE)
lcavol	x	x	x
lweight	x	x	x
age	x		
lbph	x		
svi	x	x	x
lcp			
gleason			
pgg45			

There is strong agreement between the three approaches. All of them select `lcavol`, `lweight` and `svi`, so these are the core predictors of `lpsa`. Best subset with CV and LASSO at lambda.1se choose exactly the same three-variable model. The only difference is that AIC also keeps `age` and `lbph`.

(b)

I start from the AIC model in (a), `lpsa ~ lcavol + lweight + age + lbph + svi`, and build two larger models.

- *Higher-order terms.* I add the squares of the four continuous variables except the `svi` because it was recorded as a binary(0 or 1). A squared term lets the relationship between a variable and `lpsa` be curved instead of a straight line.
- *Interaction terms.* We add all pairwise products of the five variables. An interaction such as `lweight:svi` lets the effect of `lweight` depend on the value of `svi`.

```r
# Model 1: main effects (the AIC model from part a)
fit_main <- fit_aic

# Model 2: main effects + squared terms
fit_sq <- lm(lpsa ~ lcavol + lweight + age + lbph + svi +
               I(lcavol^2) + I(lweight^2) + I(age^2) + I(lbph^2), data = prostate)

# Model 3: main effects + all pairwise interactions
fit_int <- lm(lpsa ~ (lcavol + lweight + age + lbph + svi)^2, data = prostate)
```

To decide whether the extra terms improve the model, we compare the AIC of the three models. AIC balances fit against the number of terms, and lower is better.

```
data.frame(model   = c("Main effects", "+ squared terms", "+ interactions"),
           n_terms = c(length(coef(fit_main)), length(coef(fit_sq)), length(coef(fit_int))),
           AIC     = round(c(AIC(fit_main), AIC(fit_sq), AIC(fit_int)), 2))
```

```
            model n_terms    AIC
1    Main effects       5 215.90
2 + squared terms       9 220.57
3  + interactions      15 221.45
```

Results. Adding squared terms increases the AIC from 215.9 to 220.6, and adding interactions increases it to 221.5. Neither extension improves the model: the extra terms add complexity without improving the fit enough.

Final model. The main-effects model has the lowest AIC, so we keep it as the final model:

```
final <- fit_main
summary(final)
```

```
Call:
lm(formula = lpsa ~ lcavol + lweight + age + lbph + svi, data = prostate)

Residuals:
     Min       1Q   Median       3Q      Max
-1.83505 -0.39396  0.00414  0.46336  1.57888

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.95100    0.83175   1.143 0.255882
lcavol       0.56561    0.07459   7.583 2.77e-11 ***
lweight      0.42369    0.16687   2.539 0.012814 *
age         -0.01489    0.01075  -1.385 0.169528
lbph         0.11184    0.05805   1.927 0.057160 .
svi          0.72095    0.20902   3.449 0.000854 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7073 on 91 degrees of freedom
Multiple R-squared:  0.6441,    Adjusted R-squared:  0.6245
F-statistic: 32.94 on 5 and 91 DF,  p-value: < 2.2e-16
```

The final model is `lpsa ~ lcavol + lweight + age + lbph + svi`, with adjusted $R^2 = 0.6245$. `lcavol`, `lweight` and `svi` have clear positive effects on `lpsa`, while `age` and `lbph` are weaker.